

RideSmart - Modelagem e Análise de Rotas Urbanas com Grafos

Carlos Silvano de Oliveira Junior, Félix Luiz Garção Filho, Pedro Henrique da Silva Paixão

Escola de Ciências e Tecnologia

Universidade Federal do Rio Grande do Norte

carlossilvano1319@gmail.com, felix.garcao.122@ufrn.edu.br,

pedro.paixao.709@ufrn.edu.br

Resumo—Este relatório apresenta o resultado do estudo de rotas urbanas sob a perspectiva da teoria dos grafos. O projeto emula o problema de aplicativos de mobilidade multimodal (como Uber, 99, etc.), em que o usuário que solicita o serviço pode realizar a travessia de um pequeno trecho a pé até um ponto de embarque melhor (em perspectiva de tempo ou distância, por exemplo). Para esse problema, foram implementados e comparados quatro distintos Shortest-path Algorithms, sendo três destes (Dijkstra Simples, Dijkstra com Heap e A*) algoritmos estudados dentro da disciplina e o quarto (Dijkstra Bidirecional) escolhido de forma arbitrária a partir dos algoritmos presentes na literatura para esse tipo de problema. A ideia central era comparar como cada um desses métodos se comportava e quais os resultados eram obtidos para diferentes condições de trânsito.

Index Terms—Dijkstra, A*, Grafos, Mobilidade Urbana, OSMnx.

I. INTRODUÇÃO

O roteamento eficiente em malhas viárias urbanas é um desafio clássico na ciência da computação. Com o advento de aplicativos de mobilidade, surge a necessidade de algoritmos que não apenas encontrem o caminho mais curto, mas que otimizem o tempo de viagem considerando variáveis dinâmicas, como o trânsito e ponto de embarque, por exemplo.

O RideSmart propõe uma análise detalhada dessas rotas. A premissa central consiste em avaliar se o deslocamento a pé da origem até um ponto de embarque alternativo pode resultar em uma rota global (caminhada + carro) mais rápida do que iniciar a viagem de carro exatamente do ponto em que o serviço foi solicitado.

II. MODELAGEM DO PROBLEMA

A malha viária foi extraída utilizando a biblioteca OSMnx. O mapa da cidade escolhida é então modelado como um grafo direcionado $G = (V, E)$, em que os nós (V) representam os cruzamentos e as arestas direcionadas (E) representam as vias.

Foram definidos dois pesos principais para as arestas:

- **Distância Físico-Espacial:** Comprimento real da rua em metros, utilizado para priorizar caminhos mais curtos.
- **Tempo com Trânsito Sintético:** Tempo calculado em segundos com base na velocidade da via, acrescido de penalidades para simular congestionamentos. Vias principais receberam fator multiplicativo de atraso maior, e cada cruzamento adicionou uma penalidade fixa de segundos.

Uma limitação notável dessa modelagem é que não é possível capturar o comportamento dinâmico do trânsito de qualquer cidade (como um maior fluxo e maior congestionamento em vias principais em horários de pico, ou menos trânsito em horários mais amenos) que são observados ao redor do mundo. Isso só seria possível de ser realizado através de requisições em tempo real em APIs como a do Google Maps ou Waze, porém é evidente que essa é uma requisição paga e que foge dos objetivos primários do projeto.

III. ALGORITMOS UTILIZADOS

Para a busca dos melhores caminhos, foram comparadas quatro abordagens distintas:

A. Dijkstra Simples

O algoritmo de Dijkstra simples é um método clássico da teoria dos grafos utilizado para encontrar o caminho de menor custo entre um nó de origem e todos os outros nós em um grafo que possua arestas com pesos não negativos.

Como funciona:

- O algoritmo mantém dois conjuntos de vértices: os já visitados (onde a menor distância final já é conhecida) e os não visitados.
- No início, a distância para o nó de origem é definida como 0 e para todos os outros nós como infinito (∞). Em cada iteração, o algoritmo seleciona o nó não visitado que possui a menor distância provisória acumulada.
- Em seguida, ele "expande" esse nó, verificando todos os seus vizinhos. Se a distância atual do nó somada ao peso da aresta até o vizinho for menor que a distância registrada anteriormente para esse vizinho, o valor é atualizado (processo conhecido como relaxamento).

Desempenho: Na sua implementação mais simples (usando listas ou arrays não ordenados), o algoritmo precisa varrer todos os nós não visitados a cada iteração para achar o menor valor. Isso resulta em uma complexidade de tempo de $O(V^2)$, onde V é o número de vértices (nós). Em malhas urbanas gigantes, essa varredura linear torna o processo muito lento.

B. Dijkstra com Heap

O Dijkstra com Heap não altera a lógica matemática fundamental de "relaxamento" das arestas do algoritmo

original, mas otimiza drasticamente a estrutura de dados utilizada para selecionar o próximo nó a ser expandido.

Como funciona a otimização:

- Em vez de usar uma lista simples e procurar o menor valor do zero a cada iteração, esta versão utiliza uma Fila de Prioridade baseada em uma estrutura de árvore binária chamada Min-Heap. O Min-Heap tem uma propriedade matemática de sempre manter o menor elemento no topo da estrutura (raiz).
- Toda vez que uma distância é atualizada, ela é inserida no Heap. Quando o algoritmo precisa saber qual é o próximo nó mais próximo, ele simplesmente "puxa" o topo do Heap de forma quase instantânea.

Desempenho: Extrair o menor elemento de um Heap custa apenas $O(\log V)$. Isso reduz a complexidade total do algoritmo para $O((V + E) \log V)$, onde E é o número de arestas. Como as redes de ruas das cidades são grafos "esparcos" (onde E é muito menor que V^2), o Dijkstra com Heap roda em uma fração minúscula de tempo comparado à versão padrão, tornando-o viável para aplicações do mundo real.

C. Algoritmo A*

Enquanto o Dijkstra expande seus nós de maneira concêntrica em todas as direções, o A* introduz um "senso de direção" ao processo de varredura. Isso é alcançado através da incorporação de uma função de custo total avaliada para cada nó n , definida matematicamente pela equação:

$$f(n) = g(n) + h(n) \quad (1)$$

Em que $g(n)$ representa o custo exato acumulado partindo do nó de origem até o nó atual n , e $h(n)$ consiste na função heurística informada.

A heurística implementada calcula a estimativa de tempo restante em linha reta até o destino final, utilizando a distância Euclidiana baseada nas coordenadas geográficas de latitude e longitude dos nós obtidas via OSMnx.

Para que o algoritmo A* garanta a descoberta do caminho mínimo exato, a função heurística deve ser admissível, ou seja, ela nunca pode superestimar o custo real para se alcançar o destino. Dado que a distância em linha reta (Δx) entre dois pontos geográficos é sempre menor ou igual à distância real percorrida pela malha viária sinuosa de uma cidade, a condição de admissibilidade é rigorosamente respeitada ($h(n) \leq h^*(n)$).

Geometricamente, o impacto dessa formulação matemática na eficiência do algoritmo é evidente. Ao organizar a fila de prioridades (Min-Heap) com base no valor de $f(n)$ em vez de apenas $g(n)$, o espaço de busca deixa de se expandir de forma circular uniforme e passa a se concentrar em um elipsoide direcionado com foco vetorizado em direção ao destino. No cenário prático estudado, isso se traduziu em uma redução drástica do espaço de busca: enquanto o Dijkstra expandiu mais de 7000 nós no grafo de tempo livre, o A* atingiu o mesmo caminho ótimo expandindo apenas 76 nós.

Tabela I
MÉTRICAS DE ROTEAMENTO (PONTA NEGRA → MIDWAY MALL)

Algoritmo	Dist. carro (km)	T. trânsito (min)	T. livre (min)	Caminhada (m)
Dijkstra Simples	7,31	34,4	6,6	86
Dijkstra Heap	7,31	34,4	6,6	86
A*	7,31	34,4	6,6	86
Bidirecional	7,31	34,4	6,6	86

Tabela II
NÓS EXPANDIDOS POR ALGORITMO E GRAFO DE PESO

Algoritmo	Dist.	Trânsito	T. livre
Dijkstra Simples	8368	7606	7226
Dijkstra Heap	8369	7607	7227
A*	826	5086	76
Bidirecional	70	66	71

D. Dijkstra Bidirecional

Escolhido como o algoritmo adicional da literatura, o Dijkstra Bidirecional é uma otimização robusta amplamente empregada em sistemas modernos de roteamento (como sistemas de GPS). Em vez de executar uma única busca que se expande radialmente a partir da origem até encontrar o destino selecionado, esta abordagem realiza duas buscas simultâneas: uma busca progressiva partindo da origem, e uma busca regressiva partindo do destino e caminhando no sentido inverso das arestas.

O algoritmo gerencia duas filas de prioridade independentes. A condição de parada não ocorre no exato instante em que as duas fronteiras se tocam, mas sim quando a soma das distâncias mínimas de ambas as filas se torna maior ou igual ao melhor caminho global (μ) já descoberto na intersecção.

Geometricamente, se a busca clássica do Dijkstra explora uma área proporcional a πr^2 (em que r é o raio de distância entre origem e destino), a busca bidirecional explora duas áreas consideravelmente menores, proporcionais a $\pi(r/2)^2$. Na prática, isso significa que a área total explorada cai pela metade, poupando processamento e memória consideráveis sem comprometer a exatidão do cálculo do melhor caminho.

IV. EXPERIMENTOS E RESULTADOS

Os testes avaliaram o trajeto de Ponta Negra ao Midway Mall (Natal/RN) com raio máximo de caminhada de 500 m. A Tabela I consolida as métricas de roteamento (distância percorrida de carro, tempo com trânsito sintético, tempo livre e caminhada até o ponto de embarque). A Tabela II compara o custo computacional de cada algoritmo medido pelo número de nós expandidos nos três grafos de peso (distância, trânsito e tempo livre). Em todos os casos, os quatro algoritmos convergiram para a mesma rota ótima, validando as implementações.

A. Eficiência dos Algoritmos

A análise comparativa entre as quatro implementações algorítmicas realizadas neste projeto (*Dijkstra Simples*, *Dijkstra com Heap*, *A** e *Dijkstra Bidirecional*) evidenciou como a

escolha das estruturas de dados e a abordagem de busca impactam diretamente o custo computacional em malhas urbanas de grande porte. Em termos de correteza, todos os algoritmos convergiram exatamente para o mesmo custo mínimo de tempo de viagem de carro ao avaliar os mesmos pontos de partida e chegada, validando as implementações. No entanto, o perfil de consumo de memória (mensurado pelo total de nós expandidos) e o tempo de execução de CPU apresentaram discrepâncias drásticas.

O *Dijkstra Simples*, que realiza uma busca linear de ordem $O(V)$ a cada iteração para identificar o nó com a menor distância provisória no conjunto de não visitados, demonstrou ser inviável para sistemas de tempo real. Ao rodar o ecossistema multimodal RideSmart sobre o grafo de Natal, o tempo de processamento da CPU sofreu uma degradação severa.

Em contrapartida, a introdução da Fila de Prioridades baseada em *Min-Heap* no *Dijkstra com Heap* reduziu o custo de extração do nó mínimo para $O(\log V)$. Contudo, em termos de nós expandidos, ambos os Dijkstras ainda realizam uma busca “cega”.

O A^* e o *Dijkstra Bidirecional* mitigaram essa limitação por caminhos distintos. O Dijkstra Bidirecional reduziu o espaço de busca colapsando duas franjas simultâneas (uma progressiva da origem e outra reversa do destino), fazendo com que a área total explorada caísse significativamente quando as fronteiras se encontraram na intersecção ótima μ . Já o algoritmo A^* provou ser o mais cirúrgico de todos: ao somar a função heurística admissível $h(n)$ ao custo real $g(n)$, o espaço de busca deixou de ser radial e se achatou em um elipsoide focado vetorizadamente em direção ao destino. Isso resultou no menor número absoluto de nós expandidos por execução, provando que a inserção de conhecimento geográfico poupa memória de forma ótima.

B. A Dinâmica da Caminhada (RideSmart)

A implementação da lógica do RideSmart permitiu analisar o comportamento de um sistema de transporte multimodal dinâmico, simulando o trade-off real entre o esforço físico do usuário (caminhada a pé $A \rightarrow P$) e a otimização do deslocamento do veículo de aplicativo ($P \rightarrow B$). A flexibilização do ponto de coleta revelou que permitir ao passageiro deslocar-se por algumas dezenas de metros quebra a rigidez dos algoritmos de navegação tradicionais, gerando soluções globais muito mais eficientes.

V. QUESTÕES PARA DISCUSSÃO

1. Como o problema foi modelado como grafo?

O problema foi modelado como um Grafo Direcionado (Dígrafo) utilizando a biblioteca OSMnx. A malha viária real de Natal foi capturada e convertida em uma estrutura de dados de lista/dicionário de adjacência. Essa estrutura permite representar de forma fiel as regras de trânsito reais, como ruas de mão única, conversões proibidas e retornos.

2. O que representam os nós e as arestas?

Nós (Vértices): Representam os cruzamentos de vias, bifurcações, esquinas e pontos geométricos de conexão na

malha urbana.

Arestas (Arcos): Representam os segmentos de ruas e avenidas que conectam esses cruzamentos, possuindo um sentido de direção obrigatório.

3. Quais pesos foram usados?

- Distância (*grafo_distancia*): O comprimento físico da via medido em metros;
- Tempo Livre (*grafo_tempo*): O tempo teórico (em segundos) gasto para percorrer a via baseando-se no limite máximo de velocidade regulamentar;
- Tempo com Trânsito Realista (*grafo_transito*): O tempo de viagem calibrado de acordo com uma velocidade média de horário de pico urbano (20 km/h), garantindo sincronia com ferramentas comerciais como o Google Maps.

4. Como o trânsito sintético alterou as rotas?

A inclusão do trânsito mudou drasticamente o comportamento das rotas. No cenário de tempo livre, o algoritmo tende a escolher caminhos em linha reta por vias locais. Com o trânsito pesado ativado, o algoritmo de caminho mínimo passa a penalizar severamente vias saturadas e redireciona o veículo para rotas que priorizam o fluxo contínuo (como grandes avenidas ou rodovias periféricas), mesmo que o trajeto físico se torne geograficamente mais longo.

5. Caminhar alguns metros melhorou a solução?

Sim. No experimento prático de Ponta Negra ao Midway Mall, o RideSmart identificou que caminhar 86 metros até o ponto de embarque ótimo P era suficiente para sair do gargalo local de origem e posicionar o veículo em um nó muito mais favorável. Sem essa otimização, o embarque direto no nó de origem — distante apenas 61 metros do ponto selecionado pelo RideSmart — forçaria o veículo a percorrer retornos residenciais lentos e semáforos de bairro, desperdiçando minutos preciosos no trecho de carro.

6. Em quais casos caminhar atrapalhou?

Caminhar atrapalha a solução em duas situações principais: Quando o raio máximo de caminhada (X) é excessivamente grande: O algoritmo pode sugerir que o usuário caminhe distâncias irreais (ex: mais de 1 km a pé) sob sol ou chuva para poupar uma diferença insignificante de tempo de carro, reduzindo o conforto do serviço de mobilidade. E Origens em vias de fluxo livre: Se o ponto de partida original A já estiver localizado em uma avenida de trânsito rápido e sentido favorável, qualquer caminhada para nós adjacentes apenas adicionará cansaço físico ao usuário sem gerar ganho no trecho de carro.

7. A menor distância foi também a rota mais rápida?

Não. O teste de validação comprovou que a rota baseada estritamente em menor distância física (metros) força o trajeto por dentro de bairros com muitos cruzamentos e conversões de baixa velocidade. Já a rota baseada em tempo com trânsito

priorizou o deslocamento por vias arteriais de maior fluidez, provando que, no cenário urbano, o caminho geometricamente mais curto raramente é o mais rápido.

8. O A* expandiu menos nós que o Dijkstra?

Sim, drasticamente menos. Enquanto o Dijkstra expande nós de maneira radial e concêntrica (em círculos para todas as direções a partir da origem), o A* utiliza uma função heurística baseada na distância em linha reta até o destino. Isso dá um "senso de direção" ao algoritmo, fazendo com que ele expanda nós focando quase que exclusivamente na direção geométrica do Midway Mall, reduzindo o espaço de busca e o tempo de processamento.

9. O Dijkstra com Heap foi mais eficiente que o Dijkstra simples?

Sim, em termos de complexidade de tempo. O Dijkstra simples varre linearmente ($O(V)$) todo o conjunto de nós a cada iteração para descobrir qual possui a menor distância atual. O Dijkstra com Heap utiliza uma Fila de Prioridade (Min-Heap), reduzindo esse custo de busca do menor elemento para $O(\log V)$. Embora o número de nós expandidos no pior caso possa ser similar, o tempo de CPU gasto pela estrutura com Heap é imensamente menor em grafos grandes.

10. O algoritmo da literatura trouxe algum ganho?

O algoritmo adicional implementado (Dijkstra Bidirecional) trouxe um excelente ganho computacional. Ao disparar duas buscas simultâneas — uma direto da origem e outra reversa do destino — as franjas de busca se encontram no meio do caminho. Isso faz com que a área total de nós explorados seja consideravelmente menor do que uma busca tradicional do Dijkstra que precisa caminhar sozinha por todo o trajeto.

11. Quais limitações existem na modelagem proposta?

- 1) Independência de Arestas: Na primeira versão do trânsito sintético (`random.uniform`), o trânsito flutuava quarteirão por quarteirão. Na realidade, o trânsito é regionalmente correlacionado (se a Engenheiro Roberto Freire para, todos os quarteirões dela travam em cadeia);
- 2) Custo da Caminhada Abstrato: O modelo foca apenas em minimizar o tempo que o usuário passa dentro do carro, sem ponderar o "peso psicológico" ou o tempo gasto no deslocamento a pé pelo usuário até o ponto P;
- 3) Grafo Estático: O mapa não sofre atualizações de eventos dinâmicos em tempo real, como acidentes repentinos, obras na pista ou bloqueios temporários.

12. Como o modelo poderia ser aproximado de um aplicativo real de mobilidade?

Para aproximar este projeto de um sistema comercial como o Uber ou Waze, deveriam ser implementadas as seguintes melhorias:

- 1) Função de Custo Bi-Objetivo: O algoritmo deveria otimizar uma função combinada, ponderando o tempo de carro e o tempo de caminhada multiplicados por fatores de conveniência (ex: 1 minuto andando "custa" o dobro do que 1 minuto confortavelmente sentado no carro);
- 2) Fatores de Trânsito Dinâmicos e Coletivos: Substituir as penalidades estáticas ou puramente aleatórias por matrizes de trânsito baseadas em horários históricos (matriz origem-destino por faixa de horário) e velocidade média dos próprios usuários trafegando na via;
- 3) Restrições de Segurança no Embarque: Adicionar uma camada de filtragem que impeça o ponto de embarque P de ser alocado em locais perigosos, vias de alta velocidade sem acostamento (ex: viadutos ou pontes) ou locais onde a parada de veículos seja proibida por lei.

VI. CONCLUSÃO

A análise empírica e teórica da eficiência dos algoritmos evidenciou o impacto crítico das decisões de modelagem de software. A transição do pior caso quadrático do Dijkstra Simples para a eficiência logarítmica da Fila de Prioridades com Min-Heap reforçou a necessidade de estruturas de dados otimizadas para o processamento de grandes volumes de dados geográficos em tempo real. Adicionalmente, as abordagens do Dijkstra Bidirecional e do algoritmo A* demonstraram que é possível mitigar o custo computacional de buscas cegas. O A*, em especial, destacou-se por sua capacidade cirúrgica de restringir o espaço de busca a um elipsoide direcionado, convertendo coordenadas geográficas puras em uma função heurística admissível altamente eficiente.

Em resumo, o projeto cumpriu com êxito os objetivos acadêmicos e técnicos, demonstrando que a inteligência algorítmica e a escolha consciente de estruturas de dados são ferramentas indispensáveis para a construção de soluções eficientes e inteligentes para os desafios das cidades modernas.

REFERÊNCIAS

- [1] G. Boeing, "OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks," *Computers, Environment and Urban Systems*, vol. 65, pp. 126–139, 2017.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.